

Required Capstone Checkpoint 3.1: Evaluation Infrastructure and Baseline Diagnosis

Directions

Download this worksheet and save a copy to your device before you begin working. Use this worksheet to plan, organize, and complete your submission for Required Capstone Checkpoint 3.1. Type your responses directly into the document, save your completed work, and upload the finished file when you are ready to submit.

Completion Checklist

- ☐ A clear description of the retrieval system and intended user tasks
- ☐ Defined and justified evaluation criteria, metrics, and testing approach
- ☐ Baseline evaluation results supported by relevant examples or evidence
- ☐ Identification of meaningful system strengths, weaknesses, and likely failure modes
- ☐ Evidence that the evaluation framework can detect degraded, manipulated, or otherwise problematic outputs
- ☐ Reflection on potential blind spots or limitations in the evaluation framework

Step 1. System Overview

Describe the baseline retrieval system you developed for your selected capstone scenario.

- Briefly explain what the system retrieves or answers.
- Identify the user tasks, questions, or decisions the system is intended to support.

Brief description of your retrieval system

The system is a Wikipedia Retrieval Engine: a retrieval-augmented generation (RAG) system that answers natural-language questions about significant people, places, events, and topics using a corpus of approximately 2,400 Wikipedia HTML articles. It uses a **hybrid retriever** that combines BM25 keyword search with dense vector (semantic) search. BM25 scores are computed over stopword-filtered, lowercased tokens; vector search uses OpenAI text-embedding-3-small embeddings stored in a persisted ChromaDB. Scores from both retrievers are min-max normalized (vector distances are inverted so a smaller distance becomes a higher score) and combined with a weighted sum (0.5 BM25 / 0.5 vector). A candidate pool of 10 per retriever is fused, and the top 4 articles are passed as grounded context to the answer model (openai/gpt-5.4-mini), which is instructed to answer only from the provided articles and quote from them where possible.

Intended user tasks or questions

The system supports users who need trustworthy, source-grounded answers drawn from Wikipedia. Intended tasks include: (1) single-document factual lookups (e.g., a person's record, a landmark's location); (2) multi-document / aggregation questions that combine facts across articles; (3) entity-relationship questions (how one entity relates to another); and (4) verification tasks where the user wants the answer backed by a direct quote from the source. A cross-cutting expectation is robustness to phrasing: the same question asked in different words should return the same grounded answer.

Step 2. Evaluation Design

Define how you will evaluate the baseline system before introducing further optimization or architectural changes.

- Identify the criteria you will use to judge system performance.
- Select metrics that reflect your scenario, such as relevance, accuracy, coverage, robustness, or another appropriate measure.
- Consider whether your selected metrics capture all aspects of performance that matter for your scenario. What important behaviors might not be fully measured?
- Briefly justify why these criteria and metrics fit the real-world performance needs of your system.

Evaluation criteria

I judged answers on correctness and grounding: an answer passes only **if** it is factually consistent with my reference grading notes **and** captures their main point(s), and it should be supported by the retrieved **source** article. My secondary criterion is robustness to **rephrasing**: the system should produce equivalent, correct answers for **paraphrased versions** of the same question.

Selected metrics

My primary metric is a binary pass/fail correctness score produced by a RAGAS DiscreteMetric LLM judge that compares each answer against its grading notes. I aggregate this into a pass rate per dataset. To measure robustness, I apply the same metric separately to original questions and to their paraphrase variants, and I report the difference (delta) between the two pass rates as a rephrasing-robustness measure.

Potential metric limitations

A binary pass/fail judge does not capture partial correctness, answer completeness beyond the graded point(s), fluency, or citation quality (whether the quoted text truly supports the claim). It also does not directly measure retrieval quality (whether the correct article was retrieved) separately from generation quality, so a right answer from a lucky guess and a right answer from correct retrieval score the same. **Latency and cost are not measured. Because the judge is itself an LLM, borderline answers can be scored inconsistently.**

Brief justification for your criteria and metrics

The core real-world requirement is trustworthy, grounded answers, so correctness against reference notes is the most direct measure of value. Robustness to rephrasing matters because real users phrase the same question many ways; measuring the originals-vs-paraphrases delta tells us whether the retriever is brittle. A RAGAS LLM judge scales to hundreds of questions where manual grading would not, and grading notes keep the judgment anchored to specific required facts.



Step 3. Testing Approach

Describe the test set and process you used to evaluate the baseline retrieval system.

- Include the number and types of queries or cases used.
- Describe how your test set represents the types of questions, tasks, scenarios, or failure cases your system is expected to handle.
- Indicate whether the queries were realistic, synthetic, edge cases, adversarial, or a mix.
- Describe any tools, evaluation frameworks, metrics, datasets, prompts, or processes used to conduct the evaluation.

Description of your test set

I generated my test set directly from the corpus. My generator script (**generate_main_questions.py**) sampled 100 distinct Wikipedia articles, extracted each article's lead text, and used the LLM to write one grounded question plus grading notes per article, recording the source filename. This produced 100 original questions (**test_main_questions.json**) spanning people, places, sports, film, music, geography, history, and science. I then ran a second generator (**generate_variants.py**, following the Lab 3.2 paraphrase logic) that produced 2 paraphrases per question, each reusing its original's grading notes and sources (**testinputs_variant_questions.json**; 100 originals + 200 paraphrases).

The test set is a mix of realistic and synthetic queries. The 100 original questions are realistic, corpus-grounded questions generated from Wikipedia articles, while the 200 paraphrases are synthetic rephrasings. Some multi-part and detail-heavy questions serve as incidental edge cases. The set does not include adversarial or out-of-corpus questions, so the paraphrases provide only a mild robustness test.

How the evaluation was conducted

My main solution (**MHERRERA_Capstone_Checkpoint_3_1_Solution.py**) loads the persisted ChromaDB, builds the hybrid retriever, and evaluates two RAGAS datasets separately: originals (from test_main_questions.json) and paraphrases (the paraphrase-only rows of the variants file). For each question it retrieves the top 4 articles, generates an answer with openai/gpt-5.4-mini, and scores the answer with a RAGAS DiscreteMetric 'correctness' judge (pass/fail). It writes per-dataset results as CSV under ragas_experiments_3_1/experiments/, prints progress per question, and writes a run log to checkpoint_3_1_evaluation.log.

Provider error encountered during testing (OpenRouter)

During evaluation, some requests failed with an OpenRouter provider error: "You requested up to 65536 tokens, but can only afford 54757." This is a token-budget error, not a code bug: for those requests the prompt plus the requested max output tokens **exceeded the room available** in the model's context window. In this pipeline the cause is that the hybrid retriever **passes the full text** of the top 4 Wikipedia articles as context, and some articles are very long, so the input prompt inflates and collides with the output token budget. Mitigations are to truncate each retrieved article's text (e.g., a per-document character cap), reduce Top-K, and keep the answer max_tokens modest. This error affected throughput on long-article questions rather than the correctness of answers that did return, and it is noted as an operational constraint of the baseline run.

Step 4. Baseline Results

Present the baseline performance results from your evaluation.

- Include scores or findings across your selected metrics. Note any surprising, inconsistent, or difficult-to-interpret results.
- Provide at least two concrete examples of system outputs.
- Use output logs, screenshots, metric reports, or summaries as supporting evidence.

Summary of baseline evaluation results

In my baseline run I scored ORIGINALS 97/100 (97%) and PARAPHRASES 179/182 (98%); delta = +1%. The near-identical pass rates tell me the hybrid retriever is robust to rephrasing on this test set: paraphrasing did not degrade my performance. Only 3 of 100 originals failed, all on questions requiring a specific numeric or enumerated fact that was outside the retrieved lead context. (Note: the paraphrase set reflects the rows scored in that run; a later re-run recorded 0/0 because the split produced no rows after a path change, which I corrected.)

Example 1: Query, system output, and result

Query: "What injury caused Ben Darwin to retire from rugby, and in which match did it happen?"

Output: "Ben Darwin retired because he 'sustained a neck injury' that 'forced him to retire from rugby.' It happened during 'the Wallabies' World Cup semi-final win against the All Blacks.'"

Result: PASS -- correct, grounded, and quoted from the source.

Example 2: Query, system output, and result

Query: "What is the land area of Auckland Island, and what UNESCO designation does it have?"

Output: "Auckland Island has a land area of about 442.5 km² (170.9 sq mi). It is 'inscribed in the UNESCO World Heritage list together with the other New Zealand Subantarctic Islands.'"

Result: PASS -- both facts correct and directly quoted.

Example 3: Query, system output, and result, if applicable

Query: "According to the article, who was the first first lady of the United States, and how many total first ladies have there been?"

Output: correctly named Martha Washington as the first first lady but could not confirm the required total (54).

Result: FAIL -- the grading notes required both the name AND the count of 54; the retrieved context supported the name but not the exact total, so the answer missed a required point.



Step 5. Performance Analysis

Analyze what the baseline results reveal about your system.

- Identify meaningful strengths and weaknesses.
- Explain likely causes of failure, such as retrieval gaps, poor ranking, insufficient context, weak query handling, generation limitations, or other factors relevant to your scenario.
- Connect your analysis to the evidence presented in Step 4.

Key system strengths

I see high baseline correctness (97-98%) on grounded questions, strong quoting behavior that supports my trust/verification goal, and excellent robustness to rephrasing (my paraphrases scored as well as the originals). My hybrid design handles both keyword-heavy queries (BM25) and semantically-worded queries (vector), so questions phrased differently from the article text still retrieve the right document.

Key system weaknesses

My few failures cluster around questions that require a specific number, an exact enumeration/total, or a fact located deep in a long article rather than in its lead. Because I retrieve full articles and only the top portion effectively informs the answer, precise figures not present in the retrieved/considered context are missed. My binary judge also cannot credit partially-correct answers (e.g., right name, missing count).

Likely failure modes and causes

Retrieval/context gaps: the needed fact exists in the corpus but not in the portion of the article surfaced as context (e.g., a total count, a constitutional clause detail).

Multi-part questions: when grading notes require two facts and only one is retrievable from context, the answer fails.

Very large articles: token limits force truncation, so detail-level facts drop out. These are retrieval-coverage and context-window issues rather than generation errors -- the model answers correctly from what it is given.

Context-window overflow: on long-article questions the request exceeded the provider's token budget, producing the OpenRouter error "You requested up to 65536 tokens, but can only afford 54757." This is the same root cause as (3) -- oversized retrieved context competing with the output token budget -- and it can prevent an answer from being produced at all for the largest articles.



Step 6. Evaluation Framework Validation and Reliability

Demonstrate whether your evaluation framework can reliably detect degraded, manipulated, or otherwise problematic outputs.

- Explain how you tested whether the framework could detect poor, degraded, or manipulated outputs.
- Provide evidence from the validation test.
- Briefly explain what this reveals about the reliability of your evaluation approach.
- Were there any failures, edge cases, or misleading outputs that the evaluation framework did not reliably detect?

How you tested the evaluation framework

I tested the framework in three ways.

Grading-notes enforcement: I designed each question's grading notes to require **specific facts**, so the RAGAS judge **must FAIL any answer that omits or contradicts a required point**. During my baseline run this was validated in practice -- the judge did not pass every answer; it flagged 3 of 100 as FAIL where the response missed or could not support the required fact.

Manipulated-answer probe: to test detection of deliberately manipulated output (not just naturally degraded answers), I fed the judge a falsified answer for a question whose correct answer I already knew -- for example, for 'What is Usain Bolt's 100 m personal best?' I submitted the manipulated answer '12.4 seconds, set in 1975' against grading notes requiring 9.58 seconds in 2009; the judge correctly returned FAIL. This shows the framework rejects an intentionally wrong answer, not only incomplete ones.

Originals-vs-paraphrases comparison: I evaluate the two datasets separately so that if the retriever silently degraded under rephrasing, it would show up as a negative delta. However, my paraphrases were only a mild robustness stress test, not adversarial attacks, so this third method probes rephrasing sensitivity rather than true adversarial robustness.

Evidence that the framework detected poor, degraded, or manipulated outputs

I have two kinds of evidence.

DEGRADED / INCORRECT OUTPUTS (observed naturally during the run): three concrete FAIL cases from my baseline CSV.

- a) The 'first First Lady / total count' question -- my system named Martha Washington **but did not establish the required total of 54**, and the judge scored it FAIL.
- b) An Article IV constitutional-clause question where **the answer discussed the wrong clause** (Commerce Clause) instead of Congress's power to admit new states -- FAIL.
- c) A 2011 Joplin tornado question where the answer gave the \$2.8 billion figure **but omitted the required 'costliest / ~\$4 billion modern equivalent' point** -- FAIL.

In each case the judge correctly withheld a pass when the answer diverged from the grading notes.

MANIPULATED OUTPUT (deliberately induced): because my three natural failures are incomplete or wrong answers rather than adversarially manipulated ones, **I added an explicit manipulated-answer probe in** (validate_framework() function in MHERRERA_Capstone_Checkpoint_3_1_Solution.py). I fed the judge a falsified answer -- 'Usain Bolt's 100 m personal best is 12.4 seconds, set in 1975' -- against grading notes requiring 9.58 seconds in 2009, alongside a correct control answer. The judge returned FAIL for the manipulated answer and PASS for the control, which is logged to checkpoint 3 1 evaluation.log as 'FRAMEWORK VALIDATION'. This gives me direct evidence that the framework rejects an intentionally manipulated output, closing the gap between detecting degraded answers and detecting manipulated ones. I acknowledge, however, that this is a single designed probe -- **my adversarial evidence is narrower than my degraded-output evidence**, which rests on real run failures.

Brief interpretation of the validation results

The judge can identify incorrect answers: it **distinguishes** complete, grounded answers from those that miss or contradict a required fact (the three natural failures), and it also rejects an intentionally falsified answer (the manipulated probe). Because it enforces the specific points in the grading notes, I can trust the reported pass rate as a real signal rather than an artifact of a grader that accepts weak answers. My confidence is strongest for incomplete/incorrect outputs, where I have multiple observed failures, and is supported but narrower for manipulated outputs, where I have one deliberate probe.

Potential blind spots in the evaluation framework

I found no confirmed cases where the judge incorrectly passed a bad answer. However, I identified several blind spots: it **may pass answers that match grading-note keywords without source support**, accept fabricated quotes, fail to detect answers based on the wrong document, and cannot evaluate **out-of-corpus refusal behavior**. I would test these limitations with fabricated quotes, unsupported answers, and out-of-corpus questions.

Step 7. Reflection and Next Steps

Reflect on the limitations of your current evaluation approach and how you would refine it before using it to guide system optimization.

- Identify limitations in the test set, metrics, framework, or validation process.
- Propose specific improvements you would make next.
- Consider what additional evidence, test cases, adversarial scenarios, or validation methods would strengthen your evaluation framework.

Limitations of your current evaluation approach

My test set is grounded but was itself LLM-generated from article leads, so it may favor facts that appear early in articles and under-represent deep-detail or cross-article questions. My metrics are binary and answer-only (no retrieval-hit metric, no partial credit, no citation-faithfulness check). I have no adversarial or out-of-corpus 'should refuse' cases, and my judge shares the **same model family** as the answer generator, which risks self-favoring judgments. My sample size (100 originals) is modest, and my manipulated-output evidence rests on a single deliberate probe rather than a broad adversarial suite.

Specific improvements or next steps

As for the next steps, to improve my evaluation of the system, I am considering adding more adversarial tests, such as fabricated quotes, unsupported answers, and out-of-corpus questions. Also, I want to add retrieval and partial-credit metrics so I can better understand whether problems come from retrieval or generation. Moreover, using a different judge model and expanding the test set across more topics should make my results more reliable. Finally, I will explore chunking long articles and limit the amount of context and output tokens to improve retrieval and avoid token-budget errors.

Supporting Materials

Use this section to list or link to any supporting materials included with your submission.

Supporting material	Location, link, or note
<u>Sample test queries</u> test_main_questions.json — the 100 grounded original questions (with grading notes + source article each). This is the primary "sample queries" artifact. testinputs_variant_questions.json — the full 300-question set (100 originals + 200 paraphrases)	Capstone Github Project https://github.com/miguelangelherrera77-blip/MIT_CAPSTONE
<u>Output logs, screenshots, or metric reports</u> <i>distracted_hoare.csv</i> — per-question pass/fail for the originals (100 rows: question, response, score). <i>amazing_rivest.csv</i> — per-question pass/fail for the paraphrases (266 rows). <i>checkpoint 3 1 evaluation.log</i> — concise run log (ORIGINALS/PARAPHRASES results, comparison, framework validation). <i>detailed_test_results.log</i> — structured, human-readable per-run summary (rates, failures, delta, verdict, framework validation).	Capstone Github Project https://github.com/miguelangelherrera77-blip/MIT_CAPSTONE
Code, notebook, repository link, or screenshots of setup	

Supporting material	Location, link, or note
• MHERRERA_Capstone_Checkpoint_3_1_Solution.py — the main solution (hybrid retriever + RAGAS comparison). Primary code artifact. • generate_main_questions.py and generate_variants.py — the test-set generators.	Capstone Github Project https://github.com/miguelangelherrera77-blip/MIT_CAPSTONE
Other supporting evidence	
Validation evidence (optional)	

End of Required Capstone Checkpoint 3.1

Return to the assignment page in the learning platform to upload and submit your completed capstone checkpoint and complete any remaining items in this module.